ECE551 / MIDTERM QUESTION PATTERNS

Short questions • hardware inference • simulation | Q# = supplied practice midterm

Q1: MULTIPLE-CHOICE FACTS / QUICK ANSWERS

Hardware first: assign/complete always_comb → gates/muxes; incomplete combinational assignment → latch; edge-triggered block → FF. Separate blocks run concurrently. A reg declaration alone does not imply a register.

Legacy: procedural LHS → reg; continuous/gate output → wire. **SV:** logic for a single driver; wire for resolved/multiple drivers. 0/1/x/z = low/high/unknown/undriven. Uninitialized storage is x. Equal-strength opposing drivers → x; stronger wins; z + weak drive → weak drive's value. Strength: supply > strong > pull > weak > highz.

```
and g(y,a,b); // output first
bufif1 g(y,a,en); // en=0 -> z
adder #(.WIDTH(16)) u(.a(A),.sum(S));
```

Binary vs unary: a&b bitwise; a&&b one-bit truth; &v all bits1; |v any bit1; ^v odd parity. ~v bitwise invert; !v one-bit zero test. ~|cnt → zero; &cnt → all ones/full. ==/!= may yield x; ===/!== compare x/z literally (TB). 0&x=0; 1|x=1. ?: infers mux; x select merges equal branch bits.

Q1 expected choices: weak+z→weak value; min/typ/max for both edges; high-volume Honda/special protocols→standard-cell ASIC. FPGA suits programmable parallel hardware; MCU software/peripherals; full-custom highest effort/control.

Mask/reduction answer: A5 & 0F = 05; |05 = 1. `#(1:2:3)` = min:typ:max for both edges; `#(2,1)` = rise,fall. RTL delays are simulation models, ignored by synthesis. Timescale sets units/precision.

Q5: WRITE SIGN / ZERO EXTENSION

N-bit unsigned: 0..2^N−1. Signed: −2^(N−1)..2^(N−1)−1. Use sized signed constants: 5'sd9. `$signed` changes sign interpretation, not width. Mixed signed/unsigned often becomes unsigned. Concatenations/part-selects are unsigned; cast when used in signed math/comparisons. Extend **before** arithmetic; keep carry/product bits.

```
assign vec16 = SE ?
  {{8{vec8[7]}},vec8} : {8'h00,vec8};
assign {co,S} = {1'b0,A}+{1'b0,B}+cin;
assign rotR = {v[0],v[15:1]};
assign shL = {v[14:0],in_bit};
```

N+N sum needs N+1 bits; N×M full product needs N+M. >> zero-fills; >>> sign-fills only for signed left operand. `$signed(4'b1001)>>>2` = 1110 (−2). Dropping k LSBs / signed >>>k rounds down: −65>>>6=−2; signed −65/64=−1. Negating the minimum signed value needs widening. `'0/'1` fill the destination.

ARITHMETIC BACKUP: OVERFLOW / SATURATION

Signed add overflow: same operand signs, changed result sign. Subtract: different operand signs, changed A sign. Carry is separate. 7F+01→80: OV1, carry0; FF+01→00: OV0, carry1.

```
ov_add = ~(A[N-1]^B[N-1]) &
          (S[N-1]^A[N-1]);
ov_sub =  (A[N-1]^B[N-1]) &
          (S[N-1]^A[N-1]);
// A-B = A + ~B + 1
```

Unsigned W→N: discarded bits nonzero → all-ones rail. Signed W→N: bits [W−1:N−1] must all equal sign. Check includes **new sign bit N−1**. Saturate clips; truncate wraps.

```
// unsigned 16->10
usat = |u[15:10] ? 10'h3FF : u[9:0];
// signed 16->10: [-512,511]
ssat = (!s[15] && |s[14:9]) ? 10'h1FF:
       (s[15] && ~&s[14:9]) ? 10'h200:
       s[9:0];
// signed 10->7: test d[8:6], sign d[9]
// rails 7'h3F (+63), 7'h40 (-64)
```

Test min−1/min/min+1/−1/0/max−1/max/max+1. Ex04 trap: 80−01→7F but OV must be1. 16→12 signed rails 7FF/800; test bits [14:11], sign bit15.

Q10: WHICH PARAMETER CAN BE OVERRIDDEN?

Answer parameter, preferably named #(.W(16)) for a parameter named W. localparam cannot be overridden. defparam is a fragile hierarchical override statement; backtick define is a macro, not per-instance customization. #define is C syntax.

```
module regN #(parameter int W=8)(
 input logic clk, input logic [W-1:0] d,
 output logic [W-1:0] q);
 always_ff @(posedge clk) q<=d;
endmodule
// regN #(.W(16)) u(.clk(clk),.d(D),.q(Q));
```

Q2 / Q9: DRAW THE INFERRED HARDWARE

Method: trigger → state elements → each D/hold equation → arithmetic/gates → muxes → reset and enable labels. Missing clocked branch means FF hold (feedback mux); missing combinational branch means latch. if/else-if means earlier condition wins.

```
// Q2 given code: intentionally incomplete
reg out;
always_comb
 casez({a,b})
  2'b01: out=~a;
  2'bz0: out=b;
 endcase
```

Q2 casez latch: 01: out=~a; z0: out=b; no default. Table 00→0,01→1,10→0,11→hold. **Draw latch D=b, EN=~(a&b).** Procedural out is reg; outer begin/end unnecessary (case is one statement). always_comb can warn about inferred latch; cannot fix missing assignments.

Q9 partial reset: posedge block: if(!rst_n) q1<=0; else {q1<=d; q2<=q1;}. Draw **two DFFs** with input muxes:

```
D1 = rst_n ? d  : 1'b0;
D2 = rst_n ? q1 : q2; // hold on reset
```

Reset is synchronous (rst_n absent from trigger). q2 is not reset; do not draw a simple unconditional pipeline. All NBA RHSs see pre-edge q1.

STORAGE TEMPLATES [EX08 / HW2]

```
// sync high reset
always_ff @(posedge clk)
 if(rst) q <= 0;
 else q <= d;

// async low reset + sync enable
always_ff @(posedge clk or negedge rst_n)
 if(!rst_n) q <= 0;
 else if(en) q <= d; // else hold

// reset-dominant SR / sticky flag
always_ff @(posedge clk or negedge rst_n)
 if(!rst_n) flag <= 0;
 else if(clear) flag <= 0;
 else if(set) flag <= 1;
// no last else: keeps previous flag

// actual level-sensitive latch
always_latch if(en) q <= d;
```

Q14 fix: with async-reset sensitivity, never combine synchronous clear into `if(!rst_n | clear)`. Separate reset, then `else if(clear)`, then set. A missing final else in a clocked block holds a FF; it is not a latch.

Ex08 trap: always @(clk) if(clk) q<=d misses d changes while clk high → faulty latch simulation. Use always_latch / @(*). always_ff expresses intent/restricts writers/timing; it does not insert a clock or guarantee every variable survives synthesis.

Q8 counter bug: bit_cnt=bit_cnt+1 in always_comb gives self-feedback, not a clock counter. Emit inc_bit; increment stored count in always_ff. A proper next-count equation is fine if assigned to a distinct signal then registered.

DRAWING VARIATIONS: SAME EXAM STYLE

if(en)q<=d at posedge: D=en?d:q, one FF plus hold mux. **if(load)q<=data;else if(shift)q<=next:** load mux wins over shift mux; otherwise feedback. **if(a)y=c&d;else y=c|d in comb:** AND and OR feed2:1 mux selected by a, no storage. **Same code without else:** latch EN=a; data=c&d.

Two-stage pipeline, both reset: put both q1<=0 and q2<=0 in reset branch, then q1<=d;q2<=q1. If only q1 resets (Q9), q2 gets hold feedback. **Blocking q1=d;q2=q1 in one block:** both use d; intended pipeline collapses. Reverse blocking order may preserve old stage values but can race with other processes; use <=.

```
// Q9 schematic connections, mux select=rst_n
//  d --1\
//         MUX --> DFF1 --> q1
//  0 --0/         ^clk
// q1 --1\
//         MUX --> DFF2 --> q2
// q2 --0/         ^clk    | feedback
//  ^----------------------+
```

Draw synchronous reset as D-input selection, asynchronous reset as FF reset pin. Separate assignment processes do not imply separate times; all FFs sample on the same edge.

Case facts for Q2: case exact x/z match; casez ignores z/? in either expression or item; casex also ignores x (can hide bugs). First matching item wins. case inside has item-side wildcard patterns. To remove a latch, define every path; adding an arbitrary default changes the specified hold behavior.

Q6 / Q7 / Q13: SENSITIVITY, RACES, CONTROL

Course queue: active (blocking, RHS evaluations, continuous assigns, display) → inactive (#0) → NBA LHS updates → monitor/strobe; repeat triggered activity until settled, then advance time. Full SV adds assertion/other regions.

=: evaluate/update before next statement. <=: evaluate now, update later in NBA. Within a begin/end order is fixed; independent blocks at same time have no general order. Race means multiple legal results; need not literally become x.

```
// Before edge q1=0,q2=1,d=1:
q1 <= d; q2 <= q1; // after NBA: 1,0
q1 = d;  q2 = q1;  // in this block: 1,1
```

Hand trace: write time / RHS sample / scheduled LHS update. A sampled delayed RHS is not reevaluated when its source changes.

Four delay forms at t:

`#5 q=d;` sample/update t+5; process waits.

`q=#5 d;` sample t, update t+5; process waits.

`q<=#5 d;` sample t, NBA update t+5; continue now.

`#5 q<=d;` wait, sample t+5, NBA update then.

```
// a=3,b=2,c=1 BEFORE clean event:
b <= a+a;      // b=6 now (NBA)
c <= #5 b+a;   // c=5 at +5
d <= #2 c+a;   // d=4 at +2
// Instead: b<=a+a; #5 c<=b+a;
// #2 d<=c+a; -> c=9 at +5,d=12 at +7
```

Q7 race: initial {a=0;b=1;} and initial {a=1;#5 b=0;} → at settled t5, a=0 OR1; b=0. Source order across blocks does not settle a.

Observe: display prints when reached (often old FF values); strobe prints settled values; monitor prints settled changes. stop pauses for interactive debug; finish exits batch; fatal fails/exits. #0 is inactive-region delay, not a robust race cure.

Q6 sensitivity: given stg1<=en?a^b:a|c; out<=stg1&c; use @(a,b,c,en,stg1) because NBA stg1 must retrigger. Prefer ordered blocking in always_comb. In combinational procedures include all read inputs and conditions; do not assume a self-written temporary will retrigger always_comb.

Q3 / Q4: CODE THE RING + FIX SELF-TRIGGER

```
wire n1,n2;
nand #5 g1(n1,EN,out);
not  #5 g2(n2,n1);
not  #5 g3(out,n2);
```

EN low initializes out=1 after settling. EN high enables 3 inversions. Half-period=sum delays=15; period30. Gate delays are inertial (short pulses filtered). Delayed continuous assigns also filter pulses. Delayed NBA can preserve each sampled transition (transport-style model).

```
// Blocking version does NOT self-oscillate:
always @(EN or out)
 out = #15 (EN ? ~out : 1'b1);
// Fix for this simulation model:
always @(EN or out)
 out <= #15 (EN ? ~out : 1'b1);
```

Blocking process is busy when it changes out and misses the event. NBA returns to event wait before update, so re-triggers. Hold EN low long enough first. This feedback-delay model is a simulation exercise.

Clock TB: initial clk=0; always #10 clk=~clk; at 1ns units gives 20ns period/50MHz. No event/delay in an infinite always loop can freeze time.

CUMMINGS RULES TO RECALL

Sequential FFs and latches: <=. Combinational procedures: =. A clocked update with arithmetic, e.g. cnt<=cnt+1, still uses <=. Do not mix =/<= in one block; no multiple procedural writers; no #0 assignments; use strobe for NBA outputs. Different reset/enables → separate FF groups.

Race-free TB: drive inputs at negedge for posedge DUT; check @(posedge clk); #1; with suitable timescale/clock period. Check combinational results only after settling. A valid waveform is evidence for that stimulus; a self-check and coverage make it stronger.

Delay drill: d1 at t0, d0 at t2; q<=#5 d at t0 → q1 at t5. q=#5 d also assigns1 at t5 but blocks its process. #5 q=d instead samples0 at t5.

ECE551 / MIDTERM QUESTION PATTERNS

FSM completion • debugging • shifters/edges • PWM • timing | Q# = supplied practice midterm

Q16: SPEC → BUBBLE DIAGRAM → HDL [18 PTS]

Prompt: on start send channel through SPI; wait for first done; wait one clock; start second transaction; present result and assert completion. Use state sequencing, control defaults, and a sticky flag.

Edges: reset→IDLE. IDLE --start/wrt→CHNNL; CHNNL --!done→self, --done→WAIT; WAIT --always/wrt→RESP; RESP --!done→self, --done/set_cc→IDLE. WAIT explicitly inserts one full state-cycle gap. Assume master clears done on newly accepted wrt.

```
typedef enum logic [1:0]
 {IDLE,CHNNL,WAIT,RESP} state_t;
state_t state,nxt_state;
logic wrt,set_cc;
wire done;
wire [15:0] cmd,rd16;

always_ff @(posedge clk or negedge rst_n)
 if(!rst_n) state<=IDLE;
 else state<=nxt_state;

always_comb begin
 nxt_state=state; wrt=0; set_cc=0;
 case(state)
  IDLE: if(strt_cnv) begin
   wrt=1; nxt_state=CHNNL;
  end
  CHNNL: if(done) nxt_state=WAIT;
  WAIT: begin wrt=1; nxt_state=RESP; end
  RESP: if(done) begin
   set_cc=1; nxt_state=IDLE;
  end
  default: nxt_state=IDLE;
 endcase
end

always_ff @(posedge clk or negedge rst_n)
 if(!rst_n) cnv_cmplt<=0;
 else if(strt_cnv) cnv_cmplt<=0;
 else if(set_cc) cnv_cmplt<=1;

assign cmd={2'b00,chnnl,11'b0};
assign res=rd16[11:0];
SPI_mstr16 u(.clk(clk),.rst_n(rst_n),
 .wrt(wrt),.done(done),.cmd(cmd),
 .rd_data(rd16),.SS_n(a2d_SS_n),
 .SCLK(SCLK),.MOSI(MOSI),.MISO(MISO));
```

cnv_cmplt is a procedural output (reg/logic); res12 bits, chnnl3. Flag holds until new start/reset. First read is ignored; second is used. Declare ports with commas, not semicolons between port groups; do not duplicate declarations.

Cycle trace: start accepted→CHNNL; first done accepted→WAIT; next edge accepts wrt→RESP; second done accepted→IDLE and cnv_cmplt1. Next-state logic is combinational; actual state updates only on clock/reset.

Old key fixes: state<=IDLE on reset, not state<=state; nxt_state=state, not <= in always_comb; consistent cnv_cmplt spelling; explicit16-bit read intermediate. Include reset arrow, both !done self-loops, transition conditions and wrt/set_cc labels on diagram.

Check which edge triggers each action:

```
// PRE-EDGE state/condition -> action -> NEW
// IDLE/start -> launch1           -> CHNNL
// CHNNL/done -> no new launch     -> WAIT
// WAIT       -> launch2           -> RESP
// RESP/done  -> set completion    -> IDLE
// CHNNL/!done and RESP/!done: hold state
```

ADAPT THIS FSM TO A NEW PROTOCOL

List phases and insert WAIT for each required whole-cycle gap. Identify data/count/timer/flag storage. Comparisons/done are FSM inputs; load/clear/count/shift/start are outputs. Default nxt=state and controls0; override by state; default illegal state→IDLE. Put counters/flags in FF blocks. Draw condition/output labels and stated output defaults.

Moore outputs depend on state; Mealy outputs on state+inputs. Decode can glitch; registering adds storage/latency. For a one-cycle Moore done pulse, add FINISH state, done1, then unconditional IDLE. Sticky completion uses SR-style FF. Define busy-request/stale-done behavior; a held old done flag must not prematurely complete a new operation.

Q8: COUNTER IN ALWAYS_COMB IS A BUG

Given: if(shft) bit_cnt=bit_cnt+1 in FSM comb logic. **Explain:** self-dependent comb feedback, not a clocked count. May simulate misleadingly/fail synthesis. Changing = to <= alone does not create a clock. Emit an increment enable then clock the register:

```
// always_comb: default inc_bit=0;
// selected state: inc_bit=shft;
always_ff @(posedge clk or negedge rst_n)
 if(!rst_n) bit_cnt<=0;
 else if(clr_bit) bit_cnt<=0;
 else if(inc_bit) bit_cnt<=bit_cnt+1'b1;
```

Alternative: distinct nxt_bit_cnt=bit_cnt+1, then FF bit_cnt<=nxt_bit_cnt. Different reset/enables→separate FF groups. A combinational loop/for-loop does not spend a clock per iteration.

Q11: COMPLETE THE SLIDING PATTERN MATCH

Prompt: most recent8 serial bits match AB, oldest at MSB. Use shifter+comparator, not a large FSM.

```
logic [7:0] shft_reg;
always_ff @(posedge clk or negedge rst_n)
 if(!rst_n) shft_reg<=8'h00;
 else shft_reg<={shft_reg[6:0],in_stream};
assign match=(shft_reg==8'hAB);
```

New bit enters0, old7 falls out; match reflects updated register after NBA. For a different pattern that could match reset padding before8 samples, add valid-count guard. Rotate preserves outgoing bit; shift discards it.

Q12: FIX THE RISING-EDGE DETECTOR

Course expects: async input intended, two FFs synchronize, third remembers synced value. rising=ff2&~ff3 has correct rising polarity. Legacy output wire; reset/preset stages; NBA assignments.

```
logic ff1,ff2,ff3;
wire rising;
always_ff @(posedge clk or negedge rst_n)
 if(!rst_n) begin
  ff1<=1; ff2<=1; ff3<=1;
 end else begin
  ff1<=sig_in; ff2<=ff1; ff3<=ff2;
 end
assign rising=ff2 & ~ff3;
// falling = ~ff2 & ff3
```

Preset1 for idle-high PB/UART; reset0 for idle-low. Equal stages avoid reset pulses. Sync reduces metastability risk, not debouncing/bus coherence. Stages suggest async input but do not logically prove it; synchronous inputs can be pipelined. Reverse blocking order can preserve this chain within one block; NBA prevents inter-process races and is expected style. SV logic allows a single continuous driver.

Q14: IDENTIFY PWM + REPAIR ASYNC RESET

Recognize:8-bit counter, set at0, clear at count>=mag → **PWM**, average motor/LED drive set by pulse width. Problem is if(!rst_n | clear), not missing final else.

```
logic [7:0] cntr;
logic set_pwm,clear_pwm;
always_ff @(posedge clk or negedge rst_n)
 if(!rst_n) cntr<=0;
 else cntr<=cntr+8'd1;
assign set_pwm=~|cntr;
assign clear_pwm=(cntr>=mag);

always_ff @(posedge clk or negedge rst_n)
 if(!rst_n) intensity<=0;
 else if(clear_pwm) intensity<=0;
 else if(set_pwm) intensity<=1;
```

Separate async reset from synchronous clear/set. Clear priority gives mag0→alwayslow. No final else means FF hold, not latch. Stable mag gives mag high clocks/256, f_pwm=f_clk/256; mag255→255/256, not100%. Output reacts to pre-edge count; post-edge count is incremented.

PWM hand trace, mag3: old cnt0→intensity1/newcnt1; old1→1/new2; old2→1/new3; old3→0/new4. Three high clock intervals. mag0 clears even atcnt0; mag1 gives one high interval. Explain the signal's duty fraction, period, and reset priority before worrying about exact module syntax.

Registered pattern-match variation: match<= (shft_reg==AB) in the same FF block tests OLD shft_reg and adds a clock of latency. For match registered on the edge accepting the newest bit, test `{shft_reg[6:0],in_stream}==8'hAB` instead. Reset match if storing it.

Q15: READ THE INPUT TIMING PICTURE

Given: T5ns; external launch cq0.25ns, external logic3.25ns; DUT setup0.25ns. **Answer input_delay=0.25+3.25=3.5ns.** It describes time already spent OUTSIDE the DUT relative to clk, not remaining internal budget.

```
create_clock -name clk -period 5 clk
set_input_delay 3.5 -clock clk sig
```

DUT input-to-capture logic budget=5−3.5−0.25=**1.25ns**. Wholly internal FF path, cq/setup0.25→5−0.25−0.25=**4.5ns**. Do not subtract DUT cq when path begins at input pin.

No skew/uncertainty: cq_max+comb_max+setup≤T; cq_min+comb_min≥hold. Positive capture-clock skew helps setup, hurts hold. Max path threatens setup, min path threatens hold. Input delay=externalcq+logic/wire; output delay=receiver external logic+setup; set_load accounts for capacitance. #gate delay does not constrain synthesis.

RAPID ANSWER MAP: ALL PRACTICE QUESTIONS

1: weak+z→weak value; min:typ:max both edges; HondaASIC; A5&0F=05, |05=1; output load (complete timing setup also needs clock-relative timing).

2: reg; begin/end optional; latch D=b,EN=~(a&b), holds11. **3:** NAND/NOT/NOT each#5; period30. **4:** blocking feedback misses event; use delayed NBA. **5:** replicate sign8 times when SE1. **6:** a,b,c,en,stg1 for supplied NBA comb block. **7:** a0or1 race,b0 at5.

8: count needs FF storage/increment enable. **9:** twoFFs, q1 sync-reset mux,q2 hold mux. **10:** parameter named override. **11:** shift8/compareAB. **12:** sync2+history1; rise=current&~previous; types/reset/NBA. **13:** stop interactive,finish batch. **14:** PWM,split reset/clear/set. **15:** input delay3.5ns. **16:** IDLE/CHNNL/WAIT/RESP + sticky completion.

SAME-STYLE VARIATIONS + SHORT SOLUTIONS

No else → latch? Only incomplete level-sensitive assignment. At posedge it means FF hold. Combinational defaults can cover omitted branch assignments safely.

Draw if(a)y=c&d;else y=c|d: AND and OR feed2:1 mux selected by a; no storage. **Draw posedge if(en)q<=d:** D=en?d:q into FF. **q1=d;q2=q1 in one clocked block:** both see d in that process, no intended two-stage pipeline. **q2 omitted during reset:** draw feedback, not reset-to-zero.

At t0 d1,t2 d0; q<=#5 d at0: q1 at5. #5 q=d instead samples0 at5. **Print after q<=d:** display commonly old q; strobe settled new q. #0 still precedes NBA, not a robust race fix. **Two initial blocks write same variable:** do not assume textual order; state all legal outcomes.

Timed enable FSM: OFF clears timer; QUALIFY restarts on bad input; ON only after uninterrupted good interval. Exit priority per spec. Insert WAIT state for required one-clock protocol gaps, no #delay.

Count N cycles/width:0..N−1 takesN clocks, needs ceil(log2N) bits (at least1).0..N inclusive needs ceil(log2(N+1)) and takesN+1 clocks. terminal=&cnt; zero=~|cnt. for loop bounds at elaboration create hardware, not clock cycles.

SELF-CHECK ANSWER PATTERN

```
// Combinational TB, declared inputs/outputs:
{A,B,cin}=i[8:0]; // int i=0..511
expected={1'b0,A}+{1'b0,B}+cin;
#1;
if({co,S}!==expected) $fatal(1,"mismatch");
// Sequential: drive at negedge;
// @(posedge clk); #1; check after NBA.
```

Four-bit adder has512 inputs; use int iterator (narrow counter wraps). Tests need independent expectations, rails/hold/reset/priority, and a timeout for waits. Finish successful batch tests with PASS and $finish.

SHORT CURRENT-HW BACKUP

UART8N1
idle1/start0/D0..D7LSBfirst/stop1;50MHz/9600→5208 clocks/bit, timer0..5207; RX half-bit2604 then full periods. TX done after complete stop, RX ready at stop center; sticky flags. PWM11 period2048,24.414kHz; nonoverlap64; widen duty+64 to12bits.
PID:P9×sat10pitch,I=int>>>6,D=−(rate>>>6),sat12sum; valid-gated overflow-hold integrator.

Partial-credit scan: truth table/D equations; storage and feedback; reset type; widths/sign extension; old/new values; inclusive counts; FSM defaults/reset/self-loops/output labels; pulse vs sticky flag. State requested behavior before extra syntax details.